

Experiment No. 5

Multithreading using Thread Class and Runnable Interface in Java

Title

Create Threads using Thread Class and Runnable Interface

Aim

To create and implement multithreading in Java using the Thread class and Runnable interface.

Software Requirement

JDK (Java Development Kit), IntelliJ IDEA/Eclipse/VS Code, Windows/Linux OS

Introduction

Multithreading allows concurrent execution of two or more parts of a program for maximum CPU utilization.

Multithreading in Java is a feature that enables a program to run multiple threads simultaneously, allowing tasks to execute in parallel and utilize the CPU more efficiently. A thread is a lightweight, independent unit of execution inside a program (process).

- A process can have multiple threads.
- Each thread runs independently but shares the same memory.

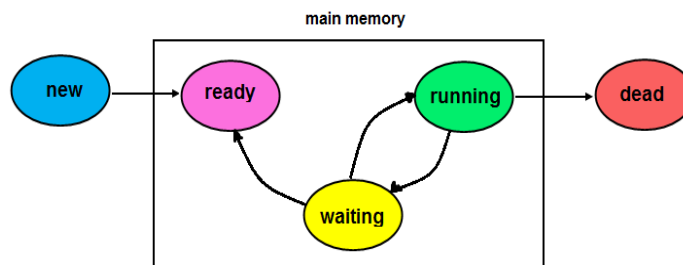


Fig.a. Threat Lifecycle

- Java doesn't support multiple inheritances, which means you can only extend one class in Java so once you extended Thread class you lost your chance and cannot extend or inherit another class in Java.
- In Object-oriented programming, extending a class generally means, adding new functionality, and modifying or improving behaviors. If we are not making any modification on Thread then use Runnable interface instead.

Theory

Java supports multithreading through Thread class and Runnable interface. Implementing Runnable is preferred for flexibility.

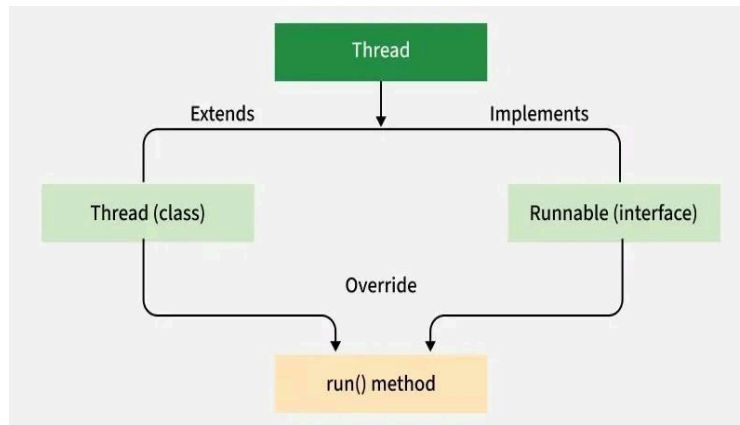


Fig.b. Thread and Runnable

When you extends Thread class, after that you can't extend any other class which you required. (As you know, Java does not allow inheriting more than one class).

When you implements Runnable, you can save space for your class to extend any other class in the future or now.

Declaration of Runnable Interface

```

public interface Runnable
{
    void run();
}

```

Declaration of a class implements Runnable Interface

```

class Sucheta_Thread implements Runnable
{
    //code
}

```

- **Runnable interface** represents a Task which can be executed by either plain Thread or Executors or any other means. so logical separation of Task as Runnable than Thread is a good design decision.
- Separating task as Runnable means we can reuse the task and also has the liberty to execute it from different means. since you can not restart a Thread once it completes. again Runnable vs Thread for task, Runnable is winner.
- Java designer recognizes this and that's why Executors accept Runnable as Task and they have worker thread which executes those task.
- Inheriting all Thread methods are additional overhead just for representing a Task which can be done easily with Runnable.
- **A thread** is a thread of execution in a program. The Java virtual machine allows an application to have multiple threads of execution running concurrently.
- The run() method contains the code executed by the thread.

Feature	Thread Class	Runnable Interface
Inheritance	Cannot inherit from any other class (Java does not support multiple inheritance).	Can inherit from other classes or implement other interfaces.
Object Type	The object <i>is</i> a thread.	The object is a <i>task</i> (runnable) that a <code>Thread</code> executes.
Flexibility	Less flexible.	More flexible and recommended for better design.

Method 1: Using the Thread Class

This approach involves creating a new class that extends the `java.lang.Thread` class and overriding its `run()` method.

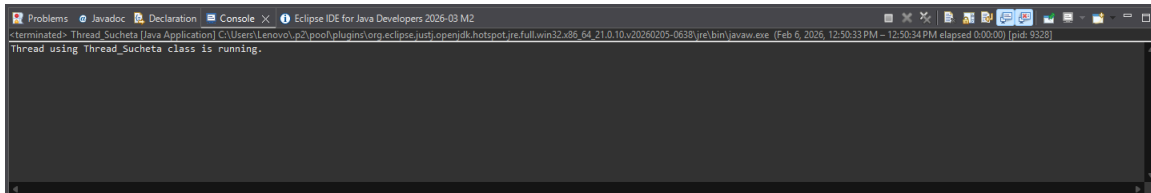
Steps:

- Define a class that extends `Thread`.
- Override the public void `run()` method with the code that defines the task the thread should execute.
- Instantiate the class.
- Call the `start()` method on the instance to begin the thread's execution. Do not call `run()` directly, as this would simply execute the code in the current thread, not a new one [1].

Source Code 1:

```
class Thread1_Sucheta extends Thread
{
    public void run()
    {
        System.out.println("Thread using Thread_Sucheta class is running.");
    }
}
// To implement in main method:
public class Thread_Sucheta
{
    public static void main(String[] args)
    {
        Thread1_Sucheta t1 = new Thread1_Sucheta ();
        t1.start(); // This starts the new thread
    }
}
```

Output 1:



Method 2: Using the Runnable Interface

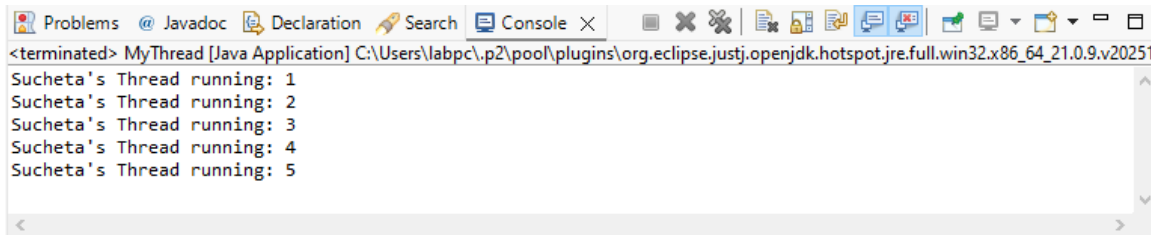
Steps:

- Define a class that implements Runnable.
- Override the public void run() method.
- Instantiate the class.
- Create a Thread object, passing the Runnable instance to the Thread constructor.
- Call the start() method on the Thread object.

Source Code 2:

```
class Sucheta2_Thread implements Runnable
{
    //The run() method contains the code executed by the thread
    public void run()
    {
        for(int i=1;i<=5;i++)
        {
            System.out.println("Sucheta's Thread running: "+i);
        }
    }
    public static void main(String[] args)
    {
        //A Thread object is created by passing the Runnable object
        Sucheta2_Thread obj = new Sucheta2_Thread();
        Thread t1 = new Thread(obj);
        //Calling start() creates a new thread and internally calls run()
        t1.start();
    }
}
```

Output 2:



Conclusion

Multithreading improves performance by executing tasks concurrently.

Viva Questions

1. What is multithreading?
2. Difference between Thread and Runnable?
3. What is thread lifecycle?